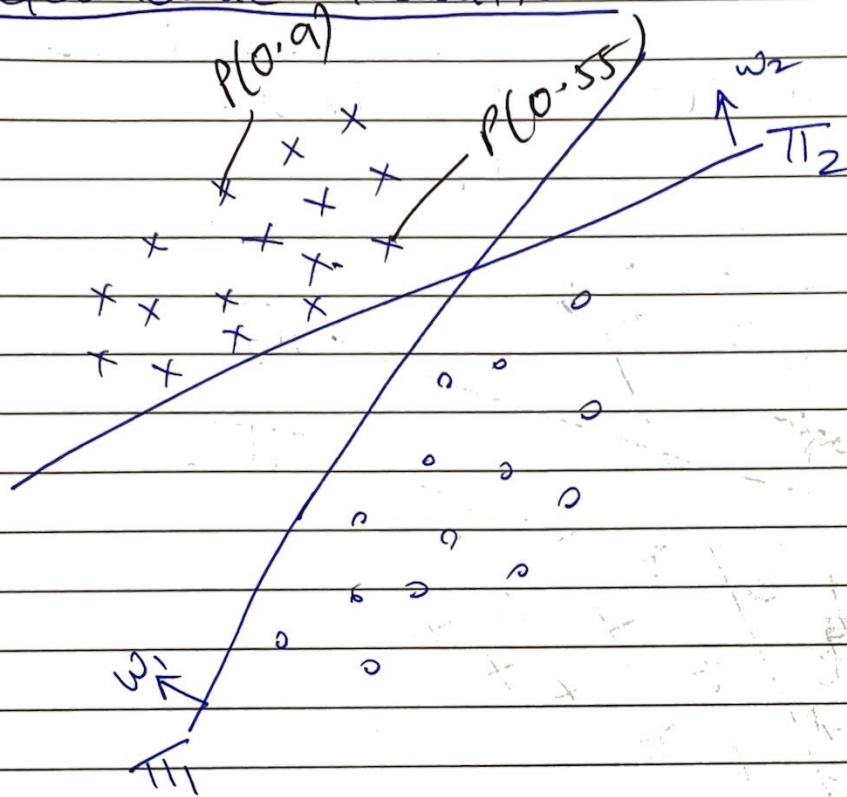


ch1 - Support Vector Machines

VI: Geometric Intuition



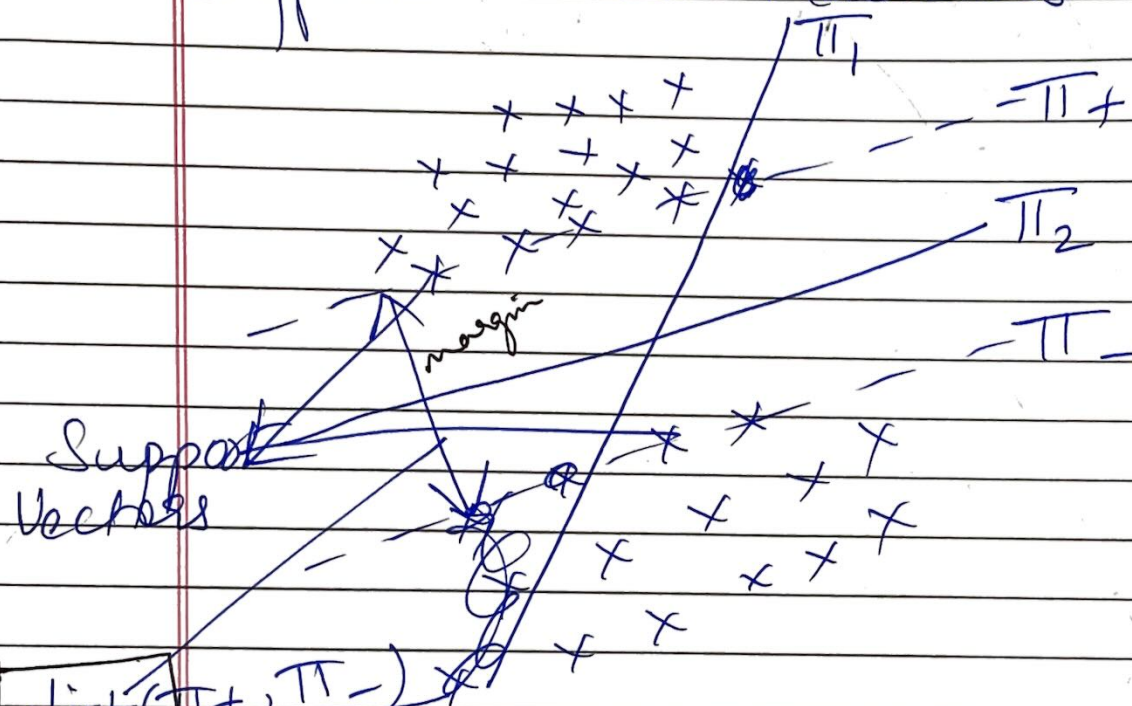
There will be many possible hyperplanes that separate positive pts. from -ve pts.

We had learnt in Logistic Regression that the probability of a point belonging to any class given z it is very close to the hyperplane will be close to 0.5.

So we want a hyperplane that separates +ve pts and -ve pts as far away as possible.

Such a hyperplane is called margin maximizing hyperplane.

The above is the key idea of Support Vector Machines.



$\text{dist}(\Pi_+, \Pi_-)$ is better than Π_1
 $\Pi_+ \Rightarrow \parallel$ to Π & touches the closest +ve point to Π .

$\Pi_- \Rightarrow \parallel$ to Π & touches the closest -ve point to Π .

SVM $\rightarrow$ Try to find a Π that maximizes the margin ($\text{dist}(\Pi_+, \Pi_-)$)

The pts through which Π_+ & Π_- pass are called Support Vectors
 As margin $\uparrow$ generalization accuracy increases.

Generalization accuracy means accuracy on unseen future data.

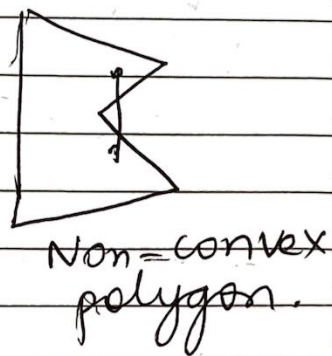
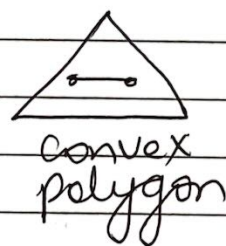
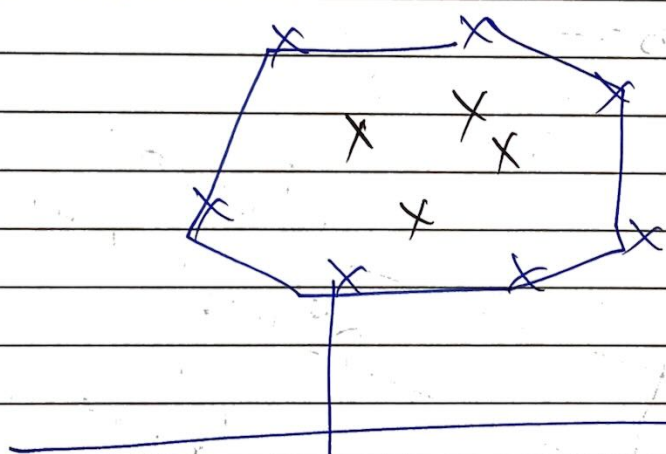
Given a new query pt. I will use hyperplane & Alternative geometric formulation of SVM for finding margin-maximizing hyperplane

1) Construct convex hull of +ve & -ve pts separately.

2) Find the shortest line connecting these hulls.

3) Bisect it

Plane that bisects it, is the margin maximizing hyperplane.



Convex hull = Smallest convex polygon such that least # of pts are boundary and rest inside.

X

class

Date
Page

V2: why we take values $+1$ and -1 for support vector plane

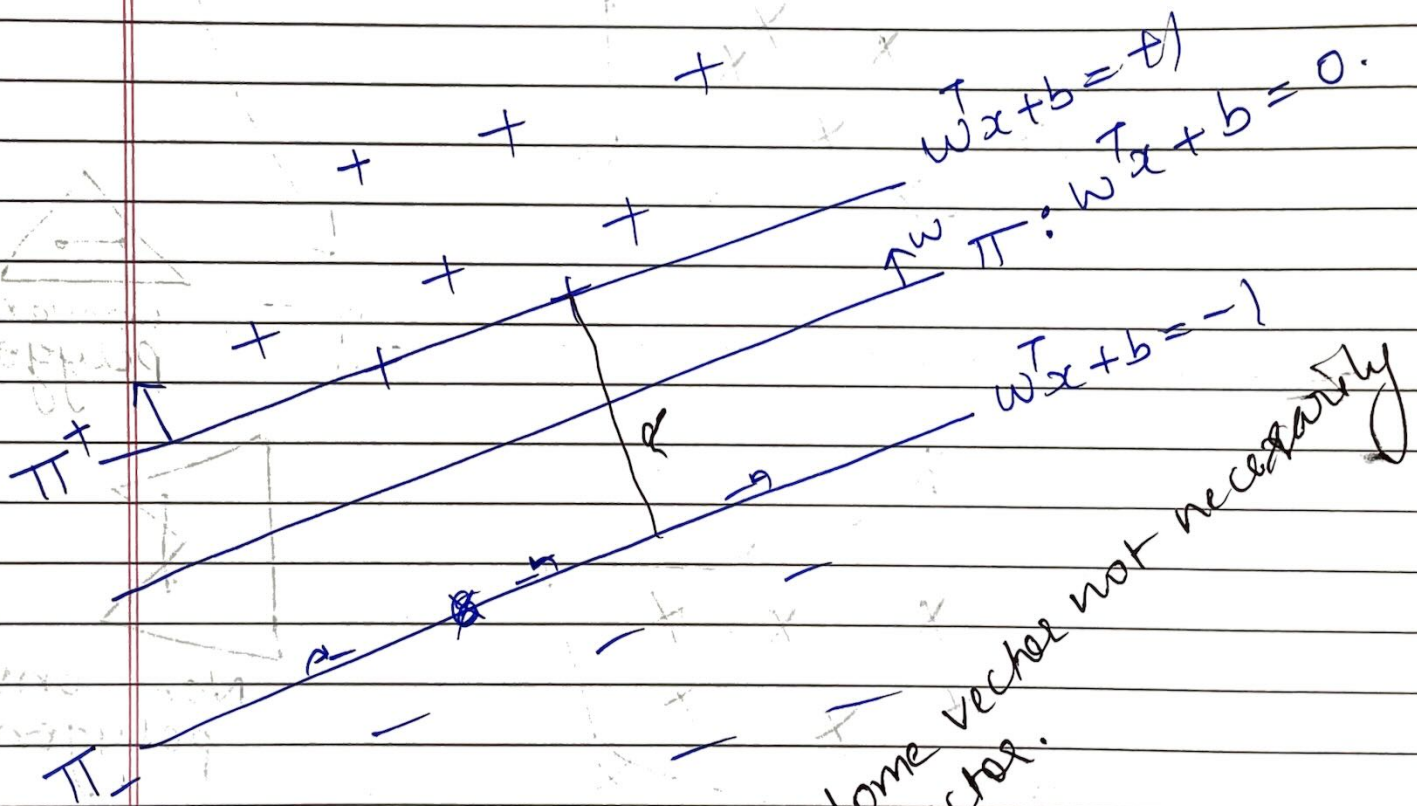
V2: Mathematical Derivation

$\Pi =$ margin maximizing

$$\Pi = w^T x + b = 0$$

if $\Pi_+ = w^T x + b = +1$

$$\Pi_- = w^T x + b = -1$$

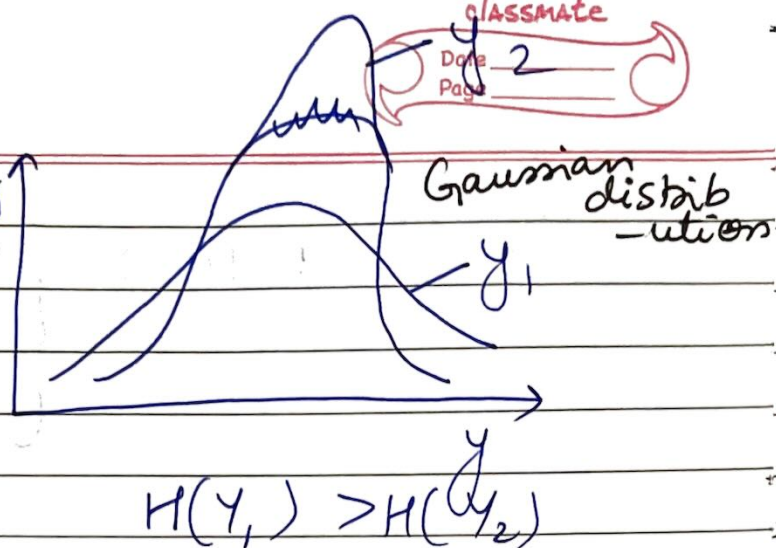
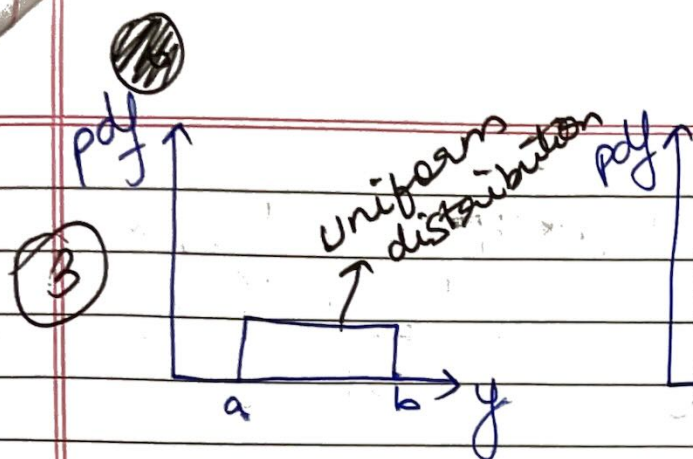


w is some vector not necessarily a unit vector.

⑥ Extending to Multiclass:-

$$Y = y_1, y_2, \dots, y_k$$

- If all are equiprobable, $H(Y) = 1$ (max)
- If any 1 is most probable & others have prob. = 0 then $H(Y) = \text{low}$.



- ④ Entropy is basically measure of random
- ⑤ ~~if the distribution is peaked, then the entropy is~~

V4: Information Gain (IG)

- Entropy of a table means basically w.r.t. the class label.
- We basically break the table on the basis of a feature, then the information gain of that feature is defined as

$$IG(\text{feature}) = H(\text{original table}) - \text{Weighted Avg } H \text{ of all the constituent tables.}$$

Let D be the original table
 $|D| = \text{No. of entries in } D.$

Let feature f_0 breaks D to D_1, D_2 & D_3 .

Then,

$$IG(f_0) = H(D) - \left\{ \frac{|D_1| * H(D_1) + |D_2| * H(D_2) + |D_3| * H(D_3)}{|D|} \right\}$$

IG plays a key role in building decision trees.

More IG of a feature means more likely is the feature in separating different classes.

More IG means the avg entropy of the constituent tables is less, i.e., randomness of the constituent tables is less.

VS: Gini Impurity

$$I_G(Y) = 1 - \sum_{i=1}^k (P(y_i))^2$$

Case I: $P(y_+) = P(y_-) = 0.5$

$$I_G(Y) = 1 - (0.25 + 0.25)$$

$$= 0.5$$

$$H(Y) = 1$$

Case II: $P(y_+) = 1, P(y_-) = 0$

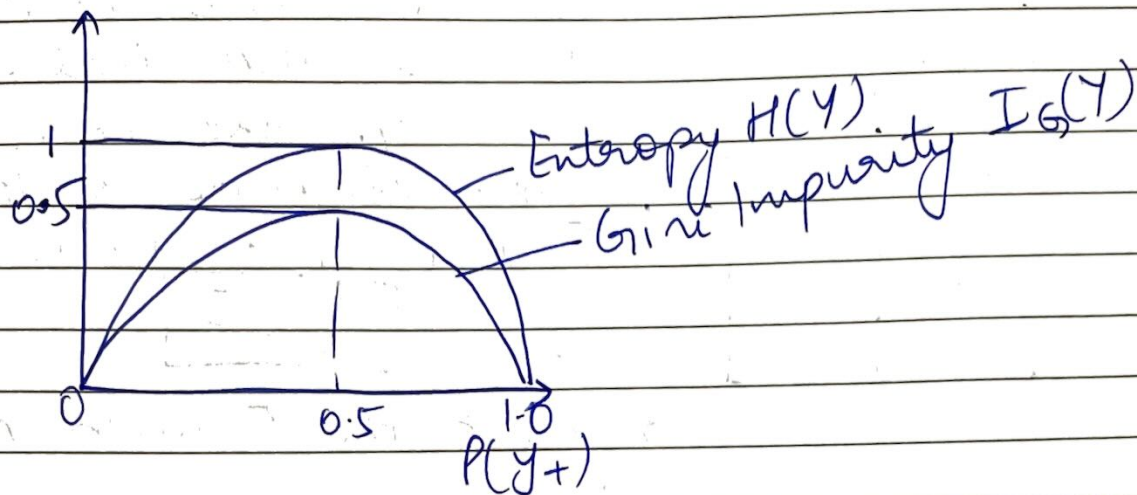
$$I_G(Y) = 1 - (1 + 0)$$

$$= 0$$

$$H(Y) = 0$$

2 category case: (y_+, y_-)

$$P(y_+) = 1 - P(y_-)$$



So, both have similar behaviours only. then why should we study $I_G(Y)$?

Computing log takes more time than computing squares.

So I_G is computationally efficient.

V6 Constructing a Decision Tree

Recursively at each level, select the feature with max. I_G to partition the table.

Eventually we may end up in one of the following cases:-

- 1) Pure node $\rightarrow$ Stop growing the node.
- 2) Very few points of other class. So further growing tree may lead to overfitting. So stop growing.
- 3) We are too deep down the tree - Stop grow. If depth is small $\Rightarrow$ underfit. DT, hyperparametering = depth.

V7: Splitting numerical features

f_1	y
2.2	1
2.6	1
3.5	0
3.8	0
4.6	1
5.3	0

- For numerical features
- 1) Sort the entries in inc. order of that numerical feature.
 - 2) Serially take each value in the feature as threshold T and split on the decision.

$$f_1 < T$$

- 3) Select the T with max. IG.

It is very time consuming as we have to go through all n entries as threshold.

V8: Feature Standardization

In DT, we don't depend on actual values of the features, but rather depend on their relative order.

So, we don't need to perform any kind of feature standardization.

V9: Categorical features with many possible values

Pincode/Zipcode — 1000s of classes.

So splitting to these many classes may cause some classes to have very few data points.

So, instead of using Picode as a categorical feature, we use a feature engineering hack to use it as a numerical feature.

We replace every P_j (Picode j) with $P(y_i = 1 | P_j)$, i.e., prob. of $y_i = 1$ given j .

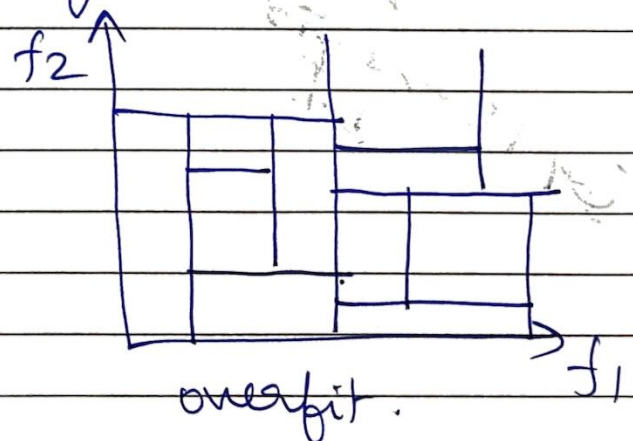
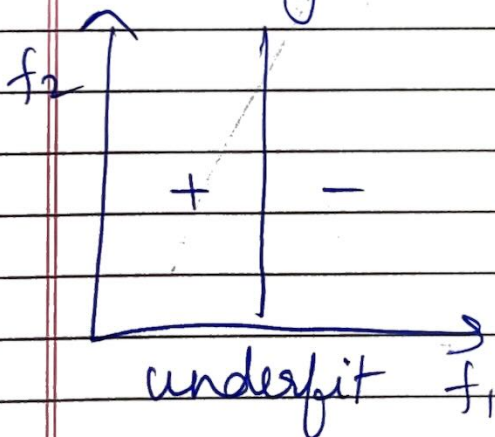
$$\frac{\# y_i = 1 \text{ and } P_j}{\# P_j}$$

So now we have an improved situation.

V10: Overfitting and Underfitting

As depth $\uparrow \rightarrow$ possibility of having very few pts at a leaf node $\uparrow$, which is sparsity. Also interpretability $\downarrow$. Due to lots of constraints.

We maybe overfitting to noise



V12: Regression using Decision Tree

Instead of entropy/Gini Impurity, we use Mean Squared Error (MSE) or MAE (Median Absolute Error).

Just like we find IG by the difference of entropy and weighted avg entropy. Here also we need to find weighted sum of MSE.

Prediction at each level $\hat{y}_i = \text{mean}(y_i)$ in that table.

In classification f_i : reducing entropy.
min. entropy = 0.

In Regression f_i : reducing MSE.
min MSE = 0.

V13 Cases

- 2es 2)
- 1) Imbalanced data - Balance it using upsampling/weights, etc.
Imbalanced data impacts entropy/MSE calculation.
 - 2) Large dimension.: At each node, split and check for max. IG feature.
So time complexity to train $\uparrow$.
For categorical feature, avoid one-hot encoding as 1 feature gets converted to many.

features. Instead convert to numerical features.

- 3) Similarity matrix - DT can't work.
Need to give features explicitly.
- 4) Multi-class classification - Naturally can be extended. Don't require one vs. Rest.
- 5) Decision Surfaces - Non linear axis parallel hyper-cuboids/spaces.
- 6) Feature Interactions - Logical feature interactions are inbuilt in DT.
- 7) Outliers - Depth $\uparrow$, outliers will impact.
Tree unstable.
- 8) Interpretability - Very much interpretable, unless depth not too much.
- 9) Feature Importance:

Normalized sum of reductions in H/I_G due to f_i - measure of feature importance.

Basically how much does f_i helps in reducing the entropy or how much does f_i helps in building D.T.

So, computing feature importance is straight forward.

By simple co-ordinate concepts we can get

$$\text{Margin } d = \frac{2}{\|w\|_2}$$

derivation in Krish Naik video. SVM part 2 in Maths folder
2 $\rightarrow$ L2 norm

So, we have to maximize 'd' with the constraint that all +ve pts. lie above Π_+ and all -ve pts. below Π_- .

Constraint Optimization Problem of SVM (Hard Margin)
So, the optimization function becomes

$$(w^*, b^*) = \underset{(w, b)}{\operatorname{argmax}} \frac{2}{\|w\|} \rightarrow \text{margin}$$

$$\text{Such that } \left[y_i * (w^T x_i + b) \right] \geq 1$$

Hard Margin SVM Constraint

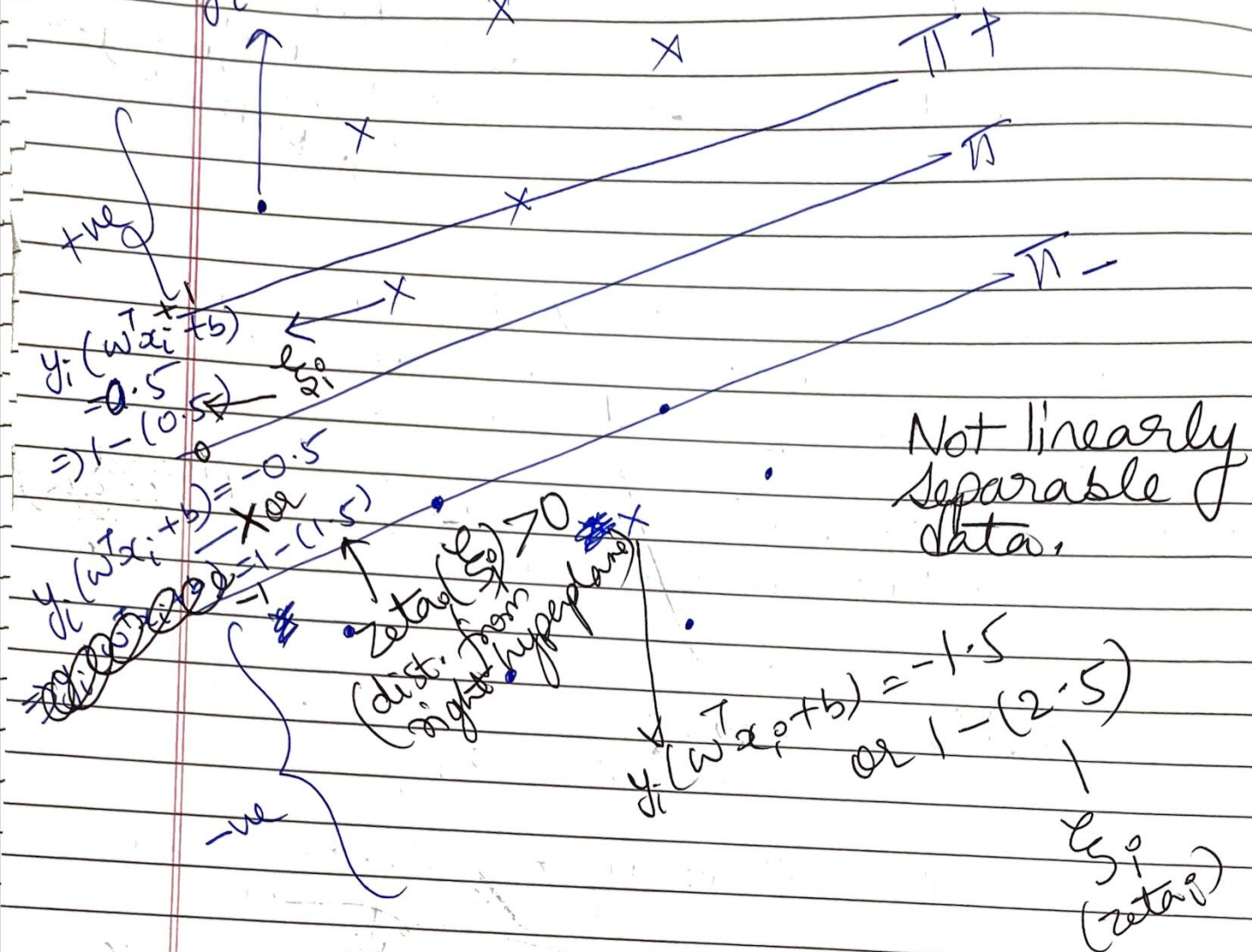
$$\forall \begin{matrix} x_i \\ y_i \end{matrix} \in D_{\text{train}}$$

This is exactly equal to 1 for support vectors

we can use Lagrange multiplier concept here.

But what if we have a dataset that's not linearly separable but almost linearly separable (overlapping)

$$y_i(w^T x_i + b) = 1.5 > 1, \quad \xi_i = 0$$



We formulate an alternate soft-margin constraint that allows some error to persist in case of non-linear data.

So we derive a new metric ξ_i for all points such that

ξ_i is the distance of the point from correct hyperplane. P.T.O

$\sum_i \xi_i = \int 0$, for correctly classified
 i.e. $y_i * (w^T x_i + b) \geq 1$
~~Correctly classified~~
 $\xi_i > 0$ & proportional to dist.
 away from that correct
 hyperplane (Π_+ or Π_-)
 for incorrectly classified
 points
 in the incorrect direction.

we are considering correctly &
 incorrectly w.r.t Π_+ & Π_-
 considered here.

This can be written in form of
 equation as ~~$\sum_i \xi_i = \sum_i \max(0, 1 - y_i (w^T x_i + b))$~~

~~$$\sum_i \xi_i = \sum_i \max(0, 1 - y_i (w^T x_i + b))$$~~

X

Now, we want to minimize these errors. (maximizing optimization problem is same as minimizing optimization)
So, our optimization function becomes

$$(w^*, b^*) \underset{w, b}{\operatorname{argmin}} \frac{\|w\|}{2} + c \left(\frac{1}{n} \sum_{i=1}^n \xi_i \right)$$

2 regularization $\leftarrow \frac{1}{\text{margin}}$

hyperparameter $\downarrow$
average distance of misclassified points

$\downarrow$
loss (called hinge loss)

$\downarrow$
constraint of the above eq.

such that $y_i * (w^T x_i + b) \geq 1 - \xi_i$ &

definition of ξ_i

$$\xi_i \geq 0$$

$$\forall i$$

note - maximizing $\frac{2}{\|w\|}$ is same as

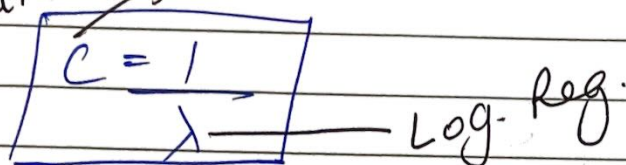
$$\text{minimizing } \frac{\|w\|}{2}$$

C-hyperparameter (Always +ve)

~~Ques~~ $C \uparrow$ = tendency to make mistakes $\downarrow$
 (Loss dominates)
 $\therefore C$ is multiplied on D_{train}
 with loss

This means overfitting which is high variance.

$C \downarrow$ = ~~underfit~~ underfit which is high bias.
 (regularization dominates) SVM



This above is called Soft margin formulation of SVM.
 In Hard Margin SVM no errors are allowed but in Soft Margin SVM errors are allowed.

V3: Why we take values +1 and -1 for Support vector planes

We have not said $\|w\| \neq 1$, which is any vector but need not be unit vector.

$$\text{margin} = \frac{2}{\|w\|}$$

1) 2)

$$\text{Let } \pi^+ = w^T x + b = +k$$

$$\text{and } \pi^- = w^T x + b = -k$$

where $k > 0$

+ - $\therefore$ we want equal gap from π

We want π^+ & π^- far away from π .

to be equally

X

So, margin = $\frac{2k}{\|w\|}$

Both are Same $\arg\max_{w,b} \frac{2}{\|w\|} \Rightarrow \arg\max_{w,b} \frac{2k}{\|w\|} = \frac{8}{\|w\|}$

where $k=4$
ie, $2 \times 4 = 8$

2) $\Pi \Rightarrow w^T x + b = k$

$$\left(\frac{w}{k}\right)^T x + \left(\frac{b}{k}\right) = 1$$

$$w'^T x + b' = 1$$

ie $w \perp \Pi$
and need not
be 1

Had it been $\|w\|=1$, $\frac{w}{k}$ wouldn't have been possible.

Reason:-

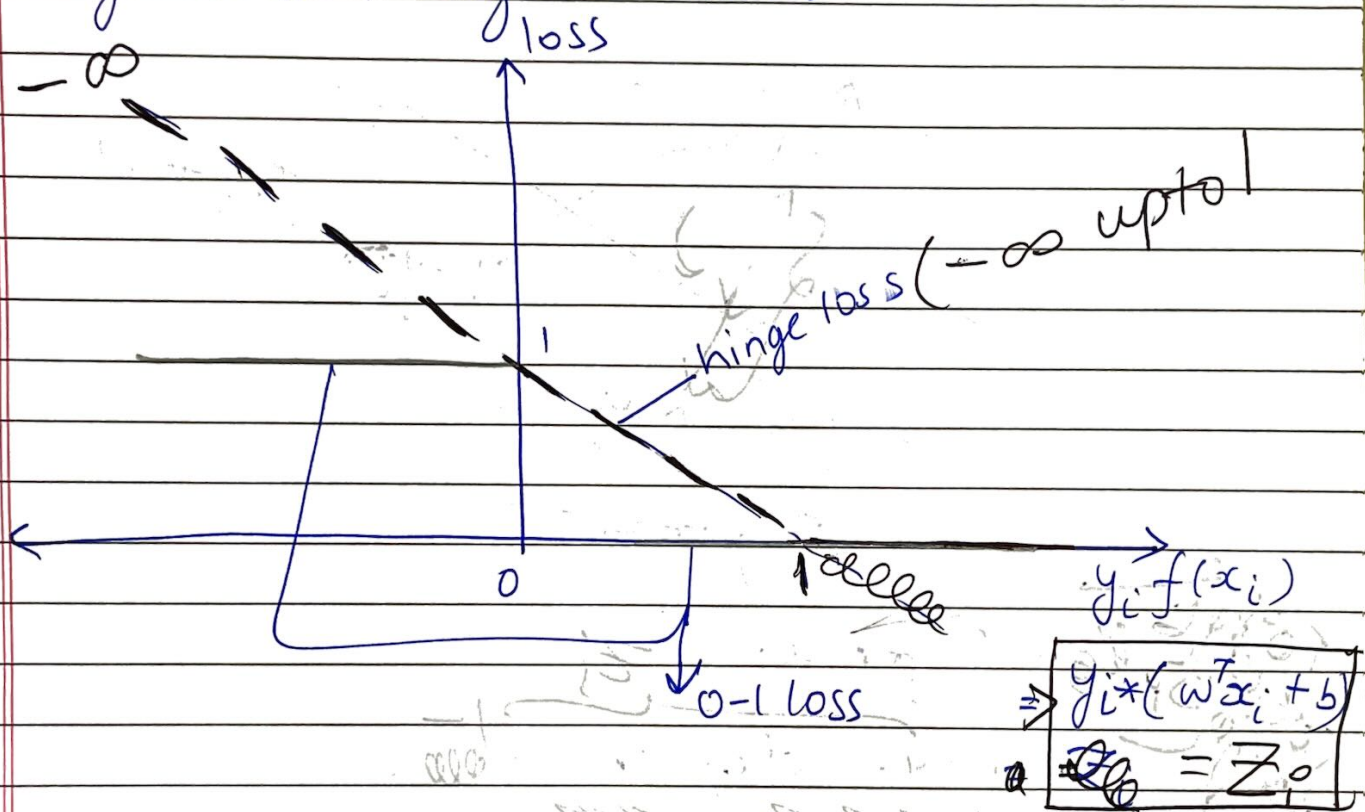
To simplify the algebraic expression,
ie, the math behind.

V4: Loss Function (Hinge Loss) based Interpretation

✓ We earlier saw that 0-1 loss is ideal loss function.

✓ But due to it being non-differentiable, we come up with other close approximate loss functions.

- 1) Logistic loss + Regularization $\rightarrow$ Logistic Regression Model.
- 2) ~~Squared~~ Linear Loss + Regularization $\rightarrow$ Linear Reg
- 3) Hinge Loss + Regularization $\rightarrow$ SVM.



Although hinge loss also isn't differentiable at $z_i = 1$, but since it is continuous unlike 0-1 loss, we can work around it.

6 min

1st form of hinge loss is not differentiable

Hinge loss

Hinge loss $\neq 0$ when $z_i \geq 1$

" $= 1 - z_i$ when $z_i < 1$

$$\max(0, 1 - z_i)$$

Case 1: $z_i \geq 1$;

$1 - z_i$ is -ve value

$$\Rightarrow \max(0, 1 - z_i) = 0$$

Note

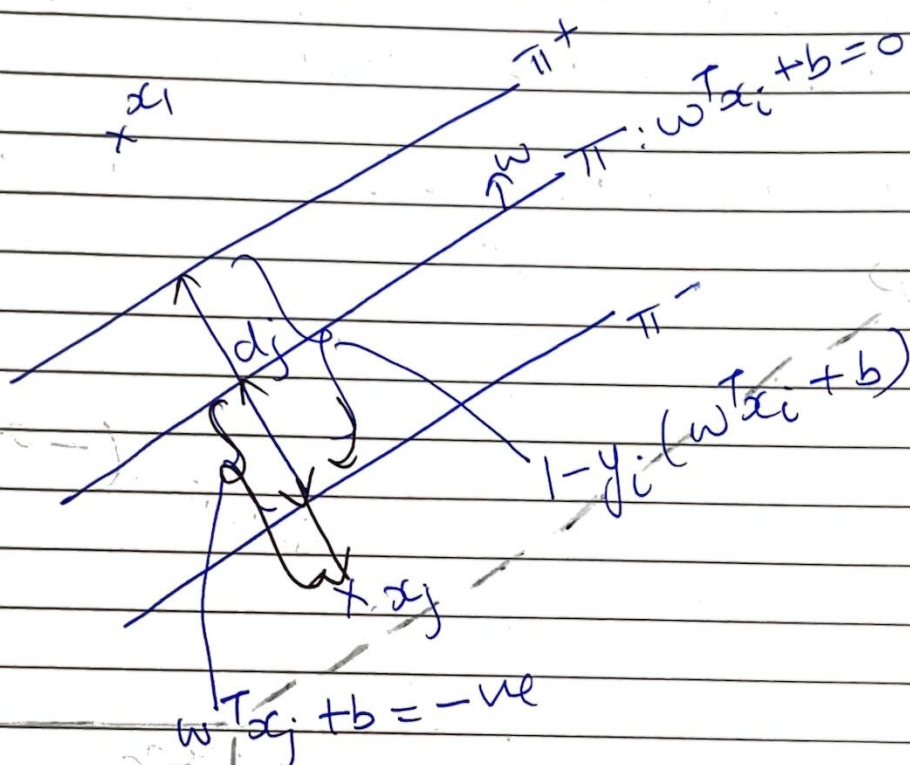
z_i is not there

$z_i \geq 0$ - x_i correctly classified points $\Rightarrow 0$

$z_i < 0$ - x_i incorrectly classified points

Case 2: $z_i < 1$; $1 - z_i > 0 \Rightarrow \max(0, 1 - z_i) = 1 - z_i$

Geometric Formulation & Loss minimization



For correctly classified point $z_i = 0$

$$d_j = 1 - y_i (w^T x_i + b) = 1 - z_j$$

z_j = distance from x_j to the $\pi^+ = d_j = 1 - z_j$

For incorrectly classified point, classmate
Date _____
Page _____

$$\xi_j = 1 - z_j \rightarrow \text{when } x_j \text{ is misclassified.}$$

10)

$$\text{Hinge loss} = \max(0, 1 - z_i) = \xi_i$$

ξ_i hyperparameter

$$\text{Soft SVM} \rightarrow \min_{w, b} \frac{\|w\|_2^2}{2} + c \sum_{i=1}^n \xi_i \rightarrow \text{loss}$$

As $c \uparrow$
overfit

As $c \downarrow$
underfit

$$\text{s.t. } (1 - y_i)(w^T x_i + b) \geq \xi_i$$

$$\xi_i \geq 0$$

and

Loss minimization $\rightarrow$ (exactly same as soft SVM)

$$\min_{w, b} \sum_{i=1}^n \max(0, 1 - y_i(w^T x_i + b)) + \lambda \|w\|_2^2$$

$\lambda \uparrow$
underfit

hyperparameter reg

$\lambda \downarrow$
overfit

Min $\frac{\|w\|_2^2}{2}$ is same as

$$\min \|w\|_2^2 \text{ in loss min.}$$

λ is put with regularization.

c is loss

$c = \frac{1}{\lambda}$	$c \uparrow \Rightarrow$ overfit		$\lambda \uparrow$ underfit
	$c \downarrow \Rightarrow$ underfit		$\lambda \downarrow$ overfit

corrections : 1:50 min (3) Sakesh
 n-clusters 1:52 min (2)
 = 30. 9:45 min (1)

338
 Date
 Page
 equivalent

V5: Dual Form of SVM Formulation

Primal Formulation Principal Form of Soft Margin SVM:-

$$\min_{w, b} \frac{1}{2} \|w\|^2 + c \sum_{i=1}^n \xi_i$$

$$\text{s.t. } y_i(w^T x_i + b) \geq 1 - \xi_i \quad \forall_i$$

$$\& \xi_i \geq 0$$

After Optimization

Dual Form of Soft Margin SVM:-

Training time $\left\{ \max_{\alpha_i} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j x_i^T x_j \right\}$ Linear SVM

$$\text{s.t. } \alpha_i \geq 0 \quad \& \quad \sum_{i=1}^n \alpha_i y_i = 0.$$

(for Support Vectors) (for non Support V)

Imp. Obs. from Dual-Form Formulation

1) For every x_i , we have an α_i

2) x_i 's only occur in the form of $x_i^T x_j$ whereas not the case in Primal SVM (x_i only)
 On solving we get the model as:-

Runtime (3)
$$f(x_q) = \sum_{i=1}^n \alpha_i y_i x_i^T x_q + b$$

- For non SV, $\alpha_i = 0$.
 So for $f(x_q) \rightarrow$ only Support V matter.

Primal SVM can be done by SMOs.
 Solving Primal SVM can be said to be equivalent to solving Dual SVMs.

④ $\alpha_i > 0$ only for Support Vectors, ~~else~~.

$\alpha_i = 0$ Non " " " "

⑤ Since x_i always occur only as $x_i^T x_j$ ^{dot product} _{Vector}

$\Rightarrow x_i \cdot x_j \Rightarrow \text{cosine-sim}(x_i, x_j)$ if $\|x_i\| = 1$; $\|x_j\| = 1$

So basically, Cosine-sim values are only required to solve the optimization & ultimately get the model.

We can replace cosine-sim with any other kind of similarity which makes it much more powerful.

Generalized Dual Form

kernel SVM $\max_{\alpha_i} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \underbrace{\text{sim}(x_i, x_j) \sim K(x_i, x_j)}_{\text{called}}$

choice)

Kernel Function can be any kind of similarity b/w x_i & x_j

5) Even during evaluation x_i occurs only as $x_i^T x_q$ $\rightarrow$ Some can use similarity values for even prediction as well.

$$f(x) = \sum_{i=1}^n \alpha_i y_i \underbrace{x_i^T x_q}_{K(x_i, x_q)} + b$$

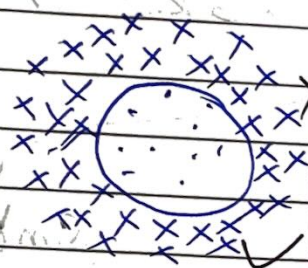
V6: Kernel Trick (Kernelization)

~~We see SVM & LR case~~

Most imp. idea of SVM is the kernel Trick.

Kernel Trick $\rightarrow$ Replacing $x_i^T x_j$ with a generalized similarity function, $k(x_i, x_j)$ called kernel function

This gives SVM potential to perform in non linear datasets.



- 1) Linear SVM - Fails. $x_i^T x_j$
- 2) Logistic Regression - Fails.
- 3) Logistic Regression + ^{Nonlinear} Feature Transform = Succeed

4) Kernel SVM + Right kernel = Succeed.

$k(x_i, x_j)$
 x_i, x_j

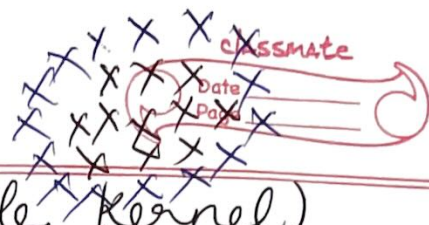
Just like in feature transformation, we map data points to a new space, similarly, the kernel in SVM does the mapping & then tries to find hyperplane in that mapped space.

~~We can but calculate~~

Results look similar:-

Linear SVM - - Does find margin-maximizing hyperplane
Logistic Regression - minimize logistic loss in x_i, x_j

Types of Kernel :-



V7 1 Polynomial Kernel (Simple Kernel)

Given circular data like above, we can
1) handle using feature transforms.

$(f_1, f_2) \xrightarrow{\text{F.T.}} (f_1^2, f_2^2)$
Apply Logistic Regression on the transformed data.
~~2) 1D Right Kernel~~

2) General polynomial kernel form - p

$$K(x_1, x_2) = (x_1^T x_2 + c)^d$$

3) A quadratic kernel can be written as:-

$$K(x_1, x_2) = (x_1^T x_2 + 1)^2 \quad \text{assuming } c=1, d=2$$

In 2D Let $x_1 = \langle x_{11}, x_{12} \rangle$ & $x_2 = \langle x_{21}, x_{22} \rangle$

$$K(x_1, x_2) = (1 + x_{11}x_{21} + x_{12}x_{22})^2$$

After expansion

$$= 1 + x_{11}^2 x_{21}^2 + x_{12}^2 x_{22}^2 + 2x_{11}x_{21} + 2x_{12}x_{22} +$$

$$2x_{11}x_{21}, x_{12}x_{22}$$

check wikipedia
(Quadratic function)

= Say $[1, x_{11}^2, x_{12}^2, \sqrt{2}x_{11}, \sqrt{2}x_{12}, \sqrt{2}x_{11}x_{12}]$: x_1

are in
spaced
a/s

$[1, x_{21}^2, x_{22}^2, \sqrt{2}x_{21}, \sqrt{2}x_{22}, \sqrt{2}x_{21}x_{22}]$: x_2

Then the product we have got due to quadratic kernel is equivalent to $\Rightarrow x_1, x_2$ instead of x_1, x_2 .
~~dot product~~ b/w 6D point transformed points.
 So the kernel basically internally let maps x_1 & x_2 to x_1' & x_2' and then finds their similarity (cosine).

Kernelization $\rightarrow d \xrightarrow[\text{internally (implicitly)}]{\text{feature transform}} d' \text{ where } (d' > d)$

In our eg, we went from 2-D data to 6-D data.
 $\rightarrow$ you can count the expanded form.

That's what Mercer's theorem says.
 (In Mathematics)

It says Kernel Trick is basically nothing but

$d \xrightarrow[\text{transform}]{\text{implicit}} d' \text{ } (d' > d)$

data not linearly separable

data linearly separable in this space.

The challenge is to select the right kernel.

Quadratic kernels can transform any conic section 2-D data such as circles, ellipses, parabola, hyperbola which could not be done in case of manual feature engineering.

V8 (2) RBF Kernel

Radial Basis Function (RBF)

It is most popular and general purpose kernel.

$$K(x_1, x_2) = \exp \left\{ -\frac{\|x_1 - x_2\|^2}{2\sigma^2} \right\}$$

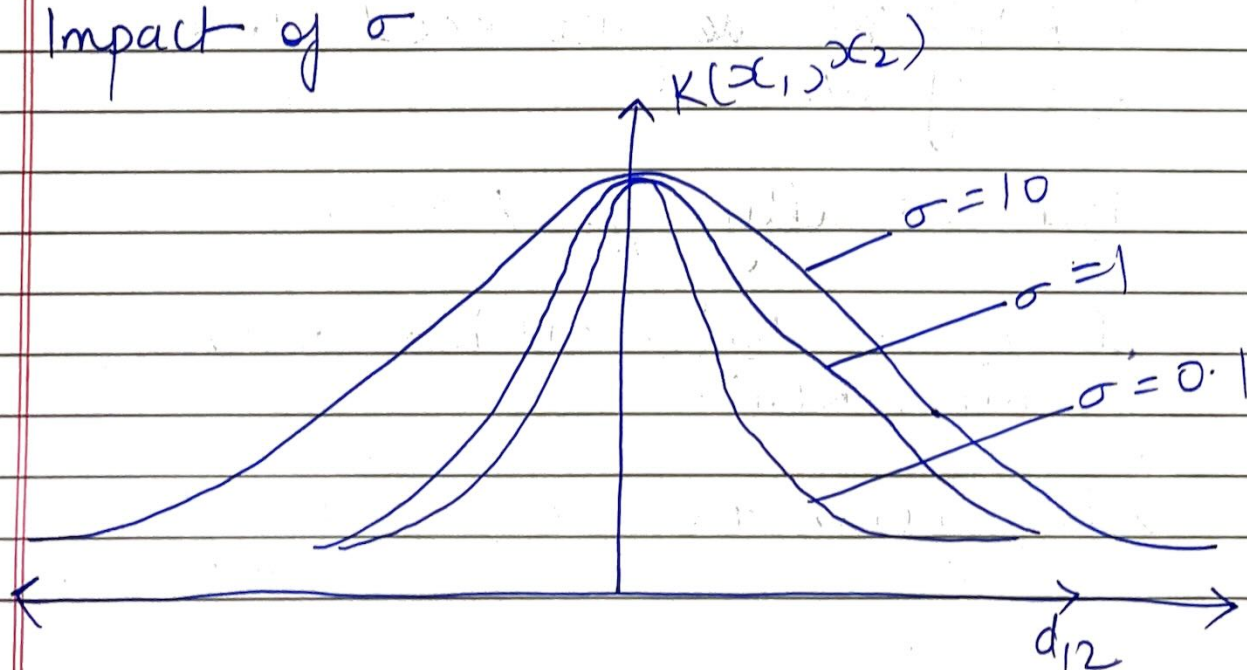
$\underbrace{\quad}_{d_{12}} \quad \underbrace{\quad}_{x_1 \quad x_2}$
 hyperparameter

$$K(x_1, x_2) = \frac{1}{e^{d_{12}^2 / 2\sigma^2}} \quad d_{12} \uparrow, K(x_1, x_2) \downarrow$$



$$K_{RBF}(x_1, x_2) > K_{RBF}(x_1, x_3)$$

Impact of σ



For small σ , the kernel value (similarity value) decreases very quickly with increasing distance.

Whereas for large σ , the kernel value decreases gradually, slowly.

So small σ is similar to small 'k' in KNN - considering very few closest neighbours.

large $\sigma \Rightarrow$ large $k \rightarrow$ considering more neighbours which are even a bit farther.

KNN works pretty good but its biggest drawback is runtime complexity.

KNN needs to store all datapoints or if optimized, LSH.

In RBFSVM, we just need to store α_i 's of SV.

And usually $\#SV \ll n$

If can't decide which kernel, go with RBF.

2 hyperparameters - C, σ - Random Search.

V9 Domain Specific kernels

With research over the years we have come up with domain specific kernels.

eg - string kernels for text classification,
genome kernels (bioinformatics)
graph kernels (for graph-data)

Domain Knowledge + Appropriate feature $\rightarrow$ transformations

Custom domain
specific kernels.

V10 Train and Runtime Complexity

Train - SGD (Stochastic Gradient Descent)

There is a specialized algo to solve dual form called Sequential Minimal Optimization (SMO)

libsvm $\rightarrow$ Best library for training SVM.
C/C++

Training Time $\div O(n^2)$ for kernel SVM.

Algorithm in 2007 $\div O(nd^2)$ if $d \leq n$. (not so popular)

If n is large, $O(n^2) \uparrow \uparrow$

So, typically don't use SVM when n is large.

It depends on $\#SV = k$ (say)

Runtime $\div f(x_q) = \sum_{i=1}^n \alpha_i y_i k(x_i, x_q) + b$

$\alpha_i = 0$ for non-SV

Then runtime complexity $= O(kd)$

$$1 \leq k \leq n$$

logistic regression runtime $O(d)$

In normal SVM we have no control over no. of support vectors

VII: nu-SVM: Control errors and Support vectors

C-SVM - original formulation.
C-hyperparameter $C \geq 0$.

nu-SVM - alternative formulation.
nu-hyperparameter $0 \leq \nu \leq 1$.

$\nu \geq$ fraction of errors
 $\nu \leq$ fraction of SVs.

eg- if we want to permit an error of 10% in worst case,
 $\nu = 0.1$.

•

Also, # of SVs is greater than 10% of n .

We can have an upper bound on

error, but not on # of SVs.

V12: SVM Regression

Math:

$$\min_{w, b} \frac{1}{2} \|w\|^2$$

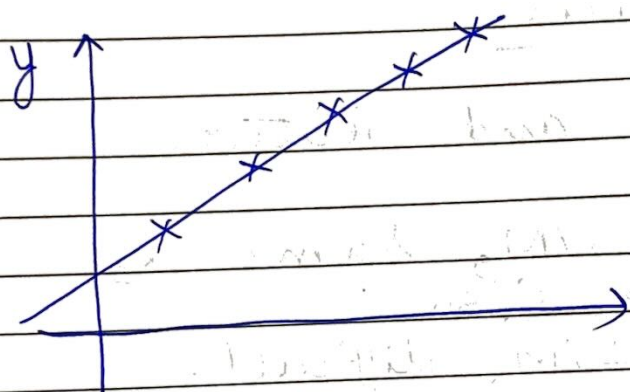
$$\text{s.t. } y_i - (w^T x_i + b) \leq \epsilon$$

$$\& \quad (w^T x_i + b) - y_i \leq \epsilon$$

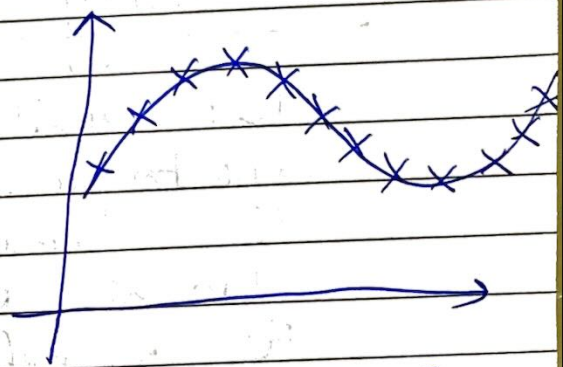
$$\epsilon \geq 0$$

epsilon hyperparameter.

This can also be kernelized.



lr-SVR



kernel SVR

$\epsilon \downarrow$ = errors low on training data
 $\Rightarrow$ overfit $\Rightarrow$ high variance.

$\epsilon \uparrow$ = errors on D_{train} higher
 $\Rightarrow$ Underfit $\Rightarrow$ High bias.

V13: Cases

Feature engineering & Transformation
→ Kernel design, finding the right Kernel.

Decision Surface → Linear SVM ~~hyperpl~~ - are

Kernel SVM → $d \ x_i$ → non linear surface

↓
 $d' \ x_i$ → linear surface

Similarity matrix

Interpretability and Feature Importance.

For linear SVM, same as Linear regression.

For kernel SVM, difficult.

Outliers → Generally v. little impact.

Exception — RBF with small σ
= KNN with small k .

Bias Variance

$C \uparrow \Rightarrow$ overfit

$C \downarrow \Rightarrow$ underfit

large $d \rightarrow$ very good for SVM

as SVMs eventually map to higher dimensions only.

Best case $\rightarrow$ Have right kernel

worst case $\rightarrow n$ is large $\rightarrow$ High train time

of SVs large $\rightarrow$ Low latency not possible